

Работа с СУБД ClickHouse в проектах СБИС

СУБД ClickHouse - это столбцовая СУБД, реализующая OLAP-задачи. Благодаря ее удачной архитектуре она зарекомендовала себя как система, позволяющая просто решать сложные статистические задачи, которые требуют анализа очень больших объемов данных. Если вы можете привести свою задачу к OLAP-реализации, то вы смело можете использовать ClickHouse и быть уверенными, что он не обманет ваших ожиданий.

Для использования работы с кластерами ClickHouse в проектах СБИС мы разработали модуль ClickHouse, который обладает простой функциональностью, но при этом вы сможете использовать функционал СУБД почти на 100%.

Из-за особенности сборки мы не можем включить данный модуль в ядро платформы, поэтому он оформлен в виде отдельного проекта, однако благодаря нашим рекомендациям вы без проблем сможете включать данную функциональность в свои проекты.

Перед тем как вы начнете использовать нашу функциональность, стоит сразу сказать, какие задачи вы не сможете решить, используя нашу реализацию.

ClickHouse не умеет:

- выполнять транзакции;
- выполнять, в стандартном понимании, операции update и delete;
- решать OLTP-задачи (типа "выдай мне всех пользователей мужчин", а вот "выдай мне, сколько пользователей мужчин" — другое дело);
- хранить большие объемы данных в одной ячейке.

Наш модуль пока не умеет:

- работать в режиме multitenancy;
- работать со сложными типами данных.

Откуда взять модуль

В состав [SBIS SDK](#) входят модули:

- DbClickhouse;
- DbClickhouse Py.

Установка и настройка модуля

Для того чтобы подключить модули DbClickhouse, DbClickhouse Py из SDK в проект, нужно создать в облаке кластер ClickHouse с именем **Статистика облака.ClickHouseTCP.Адрес** для вашего сервиса. В нем указываются адреса машин кластера и параметры подключения к нему.

На локальном стенде нужно дописать параметры в файл **service/sbis-rpc-service.ini** развёрнутого проекта:

```
1 [clusters.Статистика облака.ClickHouse.TCP]
2
3 Адрес=[{"master":"clickhouse-ch1:9000 login=Viewer
4         password=Viewer1234"},
        {"master":"clickhouse-ch2:9000 login=Viewer
        password=Viewer1234"}]
```

API модуля

Все функции для работы с ClickHouse расположены в пространстве имён **sbisdbclickhouse** на Python и C++.

Чтение

Для работы с СУБД ClickHouse нужно получить подключение методом **GetConnectedDatabase()**, а дальше работать с ним как с любой другой БД через `SqlQuery()` или `IStatement`.

Python:

C++:

Экранирование символов на ClickHouse не совпадает с экранированием на PostgreSQL, поэтому нельзя использовать общие методы `SqlIdentifier`, `SqlLiteral`.

Требуется использовать специализированные функции для экранирования `Escapeldentifier` и `EscapeLiteral` из классов `IDatabase` и `IStatement` для ClickHouse.



SQL-запросы в ClickHouse несколько отличаются от запросов к PostgreSQL. Детально можно ознакомиться с ними в [документации Яндекса](#).

Запись

Для вставки с бизнес-логики рекомендуется использовать класс **TableCopier**.

Python:

```
1 db = sbisdbiclickhouse.GetConnectedDatabase()
2 format = sbisdbiclickhouse.TableFormat(db, "default.test_table")
3 tc = db.CreateTableCopier(format)
4 rec = Record(format)for i in range(0, 10):
5     rec['field1'] = value1
6     rec['field2'] = value2
7     rec['field3'] = value3
8     tc.AddRecord(rec)
9 tc.Flush()
```

C++:

```
1 auto db = dbi::clickhouse::GetConnectedDataDB();
2 auto copier = db->CreateTableCopier( CreateRecFormat( L"test_table" )
3 );
4 *copier << value1 << value2 << value3;
5 copier->Flush();
```



По причине того, что ClickHouse плохо обрабатывает частые вставки, через `SqlQuery()` **нельзя** сделать вставку следующим образом:

```
1 SqlQuery(db, "INSERT INTO default.test_table VALUES($1,$2,$3)",
2 value1, value2, value3)
```

Но можно переписать данные из одной таблицы в другую:

```
1 SqlQuery(db, "INSERT INTO default.copy SELECT * FROM
2 deafuld.orig")
```

Удаление данных и модификация таблиц

Запросы на удаление данных/модификацию таблиц выполняются через `ALTER TABLE`. Чтобы избежать проблем с их репликацией, укажите параметр `ON CLUSTER`.

Python:

```
1 db = sbisdbiclickhouse.GetConnectedDatabase()
2 SqlQuery(db, "ALTER TABLE default.test_table ON CLUSTER test_cluster
```

```
DELETE WHERE field1=1")
```

C++:

```
1 auto db = dbi::clickhouse::GetConnectedDataDB();  
2 SqlQuery( *db, L"ALTER TABLE default.test_table ON CLUSTER test_cluster  
DELETE WHERE field1=1"_sv)
```